UNIVERSIDADE FEDERAL DE ITAJUBÁ – UNIFEI
Instituto de Engenharia de Sistemas e Tecnologia da Informação - IESTI
Engenharia Eletrônica - ELT

Sistema de Visão Computacional Embarcado para Reconhecimento de Cartas e Avaliação de Cenários do Pôquer

Aluno: Felipe Emanuel Pereira Gonçalves – d2019005551@unifei.edu.br
Orientador: José Alberto Ferreira Filho – jose.alb@unifei.edu.br
Co-Orientador: Marcelo José Rovai – rovai@unifei.edu.br

Itajubá – MG
2026

Sistema de Visão Computacional Embarcado para Reconhecimento de Cartas e Avaliação de Cenários do Pôquer

ABSTRACT: This work presents an embedded system that recognizes playing cards and evaluates poker scenarios entirely on a Raspberry Pi 5. A YOLOv8 Nano detector identifies the 52 card classes (mAP50 of 0.994 on the test set), a deterministic hand evaluator resolves conclusive situations through a fast path, and the Gemma 3 1B language model with Retrieval-Augmented Generation handles ambiguous decisions. A Flask interface validated the pipeline: detection takes about 0.1 s, fast-path decisions are immediate and RAG-based decisions take 17–29 s. In a batch benchmark of 40 situations against a Monte Carlo equity reference, retrieval from the book lowered agreement from 55% to 30% and nearly doubled the decision time; on 40 new situations, a curated knowledge base was 30% faster than the book and the automatic correction of incoherent actions raised the hybrid system to 52.5% agreement, with 10% serious errors.

KEYWORDS: Edge AI; Computer Vision; Small Language Models; Retrieval-Augmented Generation.

RESUMO: Este trabalho apresenta um sistema embarcado que reconhece cartas de baralho e avalia cenários de pôquer inteiramente em uma Raspberry Pi 5. Um detector YOLOv8 Nano identifica as 52 classes de cartas (mAP50 de 0,994 no conjunto de teste), um avaliador determinístico de mãos resolve situações conclusivas por um caminho rápido e o modelo de linguagem Gemma 3 1B, com Geração Aumentada por Recuperação, trata as decisões ambíguas. Uma interface Flask validou o fluxo: a detecção leva cerca de 0,1 s, decisões pelo caminho rápido são imediatas e decisões com RAG levam de 17 a 29 s. Em um benchmark com 40 situações, comparado a uma referência de equidade por Monte Carlo, os trechos do livro reduziram a concordância de 55% para 30% e quase dobraram o tempo de decisão; em 40 situações novas, uma base curada foi 30% mais rápida que o livro e a correção automática de ações incoerentes levou o sistema híbrido a 52,5% de concordância, com 10% de erros graves.

PALAVRAS-CHAVE: IA na borda; Visão computacional; Modelos de linguagem compactos; Geração aumentada por recuperação.

1 INTRODUÇÃO

A Inteligência Artificial na borda (*Edge AI*) desloca a execução de modelos da nuvem para dispositivos próximos à fonte dos dados, reduzindo a latência, a dependência de conectividade e a exposição de informações sensíveis [1]. Dispositivos eletrônicos como a Raspberry Pi 5 tornaram viável executar localmente tanto modelos de visão computacional quanto modelos de linguagem compactos, desde que se respeitem as restrições de memória, processamento e energia desse tipo de hardware [2].

O pôquer é um ambiente clássico de pesquisa em IA por envolver informação imperfeita e tomada de decisão sob incerteza [3], [4]. Sistemas como o Libratus atingiram desempenho super-humano [5], mas dependem de grande capacidade computacional e de entradas digitais. Neste trabalho, o jogo é usado como cenário controlado para um problema diferente: perceber a mesa física por meio de uma câmera, o que exige detectar simultaneamente múltiplas cartas com oclusão parcial, e transformar essa percepção em uma recomendação interpretável, sem sair do dispositivo.

Modelos de linguagem compactos (*Small Language Models*, SLMs), como os da família Gemma 3 [6], são executáveis em CPUs embarcadas, mas tendem a produzir justificativas imprecisas quando o raciocínio exigido foge do seu conhecimento. A Geração Aumentada por Recuperação (RAG) mitiga esse problema ao fornecer ao modelo trechos de uma base de conhecimento externa, sem retreinamento [7]. Trabalhos recentes com SLMs na Raspberry Pi 5 mostram, porém, que a inferência do modelo domina a latência e que uma camada determinística é necessária para validar saídas probabilísticas [8].

Este projeto apresenta e avalia um sistema embarcado que integra detecção de cartas com YOLOv8, avaliação determinística da mão, RAG com o modelo Gemma 3 1B e uma interface web de validação, tudo executado localmente na Raspberry Pi 5. Ele dá continuidade a uma primeira etapa, desenvolvida em notebook Jupyter, na qual o detector foi treinado e a camada de decisão com modelo de linguagem foi prototipada; ao longo do texto, "primeira etapa" refere-se a esse protótipo. As contribuições são: (i) um fluxo completo da imagem à recomendação estratégica, cobrindo pré-flop, flop, turn e river; (ii) uma camada de decisão híbrida, em que um caminho rápido determinístico evita o modelo de linguagem nos casos conclusivos e uma verificação de coerência sinaliza respostas contraditórias; e (iii) a avaliação do detector em treino, validação e teste e a comparação de SLMs e de fontes de conhecimento para o RAG no hardware alvo. As Seções 2 a 5 apresentam, respectivamente, a proposta, o desenvolvimento, os resultados e as conclusões.

2 PROPOSTA

O objetivo geral é desenvolver um sistema embarcado de visão computacional e apoio à decisão que reconheça as cartas de uma mão de pôquer Texas Hold'em, avalie o cenário visível da mesa (cartas do jogador, cartas comunitárias e posição, sem considerar oponentes, pote, fichas e apostas) e recomende uma ação (Fold, Call/Check ou Raise) com justificativa, mantendo todo o processamento na Raspberry Pi 5.

O princípio do projeto adotado é separar o raciocínio determinístico do probabilístico. Tudo o que pode ser calculado com exatidão, como a categoria da mão, os projetos de flush e de sequência ou a classificação pré-flop, é resolvido por código; o modelo de linguagem é acionado apenas quando essa avaliação não é conclusiva, e sua resposta é conferida antes de ser exibida. A Figura 1 resume a arquitetura, dividida em um ambiente de treinamento na nuvem, usado uma única vez, e um ambiente de execução local.

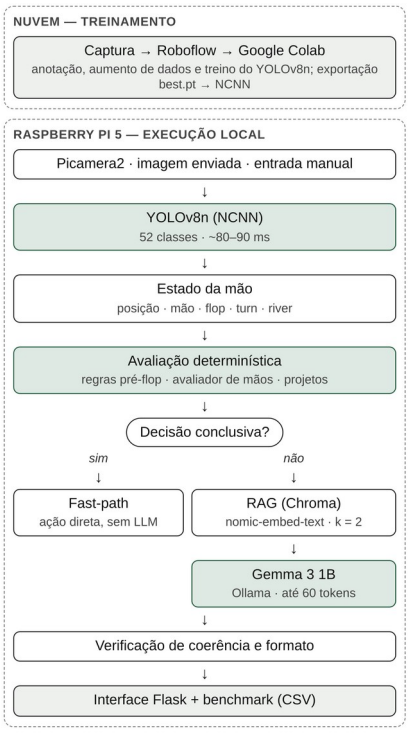


Figura 1 – Arquitetura do sistema proposto.

A Tabela 1 lista os componentes de hardware e software. O modelo YOLO é exportado para o formato NCNN, otimizado para CPUs ARM, e os modelos de linguagem e de *embeddings* são servidos localmente pelo Ollama.

Componente	Especificação
Placa	Raspberry Pi 5, Cortex-A76 4 núcleos 2,4 GHz, 8 GB
Câmera	Módulo com sensor OV5647, via Picamera2
Sistema	Raspberry Pi OS 64 bits; Python 3.13
Detecção	Ultralytics 8.4 (YOLOv8n) exportado em NCNN
Linguagem	Ollama + Gemma 3 1B
RAG	LangChain, Chroma e nomic-embed-text
Interface	Flask (aplicação web local)
Treinamento	Roboflow e Google Colab (GPU)

Tabela 1 – Componentes de hardware e software.

3 DESENVOLVIMENTO

3.1 Conjunto de dados

As imagens foram capturadas com a própria câmera da Raspberry Pi, além de capturas com a câmera de um celular, sob diferentes condições de iluminação, ângulos, sombras e oclusões parciais (cartas sobrepostas), com [número] baralho(s). A anotação das caixas delimitadoras, discriminando valor e naipe, e o aumento de dados (rotação, brilho e contraste) foram feitos na plataforma Roboflow. O acervo reúne 2.551 fotos originais e 20.714 cartas anotadas, com cerca de 150 fotos e 400 instâncias por classe. As fotos foram divididas em 82% para treino, 11% para validação e 7% para teste, sem fotos em comum. Com três versões de cada foto de treino geradas por aumento de dados, o treino ficou com 6.285 imagens (93% do total), e a validação e o teste, com 283 e 173 fotos originais (Apêndice C).

3.2 Treinamento e escolha do detector

Os modelos foram treinados no Google Colab com imagens de 640 × 640 pixels e lotes de 16. Além do YOLOv8 Nano [9], adotado desde a primeira etapa, foram treinadas as versões YOLO11n [10] e YOLO26n [11] para comparação, com 20 épocas e avaliação no conjunto de validação. O YOLOv8n foi então treinado por 50 épocas e avaliado nos três conjuntos (Tabela 2).

Modelo	Ép.	Conj.	P	R	mAP 50	mAP 50-95
YOLOv8n	20	Val.	0,982	0,971	0,991	0,973
YOLO11n	20	Val.	0,985	0,980	0,991	0,981
YOLO26n	20	Val.	0,913	0,887	0,962	0,950
YOLOv8n	50	Tr.	0,991	0,990	0,994	0,971
YOLOv8n	50	Val.	0,988	0,986	0,992	0,982
YOLOv8n	50	Teste	0,990	0,984	0,994	0,984

Tabela 2 – Métricas das versões do YOLO por conjunto de imagens (Tr.: treino; Val.: validação).

Com 20 épocas, o YOLO11n foi igual ou superior ao YOLOv8n em todas as métricas (mAP50-95 de 0,981 contra 0,973), enquanto o YOLO26n ficou abaixo nas quatro métricas e apresentou mais confusões com o fundo na matriz de confusão. O YOLOv8n foi mantido por três razões: a diferença para o YOLO11n é pequena diante do desempenho já obtido; é o mesmo modelo da primeira etapa, já integrado ao fluxo e validado na exportação para NCNN; e, após 50 épocas, atingiu mAP50 de 0,994 e mAP50-95 de 0,984 no conjunto de teste, com detecção de cerca de 0,1 s na Raspberry Pi (Seção 4.2). No teste, apenas 6 das 602 cartas (1,0%) foram trocadas por outra classe (Figura 7, Apêndice C). Como o teste vem do mesmo acervo, capturas novas da mesa não foram avaliadas.

3.3 Execução na borda

O modelo exportado em NCNN é carregado uma única vez na inicialização. A câmera também é aberta uma única vez, em um modo de captura de 640 × 480 pixels, e cada quadro vai diretamente da memória para o detector, eliminando o aquecimento de cerca de 2 s por captura usado na primeira etapa. Detecções repetidas da mesma carta são unificadas, mantendo a de maior confiança, já que um baralho não possui cartas duplicadas.

3.4 Avaliação determinística da mão

No pré-flop, as duas cartas são normalizadas para uma das 169 mãos canônicas (por exemplo, 88, AKs ou 72o) e consultadas em uma base de regras em JSON organizada por categorias, como "fortes" e "lixo_marginal". A partir do flop, um avaliador escolhe a melhor combinação de cinco cartas entre as do jogador e as comunitárias (até 21 combinações) e a classifica em nove categorias, de Carta Alta a Straight Flush. O avaliador também identifica o tipo de par (*top pair*, *overpair*, *underpair*), se a trinca é uma opção (par na mão + carta da mesa), as possibilidades de flush (9 *outs*) e de sequência (8 ou 4 *outs*) e se a força está inteira na mesa, situação em que as cartas do jogador servem apenas de desempate. Esse módulo substituiu a lógica da primeira etapa, que reconhecia apenas pares e trincas, e foi verificado com testes automatizados.

3.5 Escolha do modelo de linguagem

Foram avaliados, via Ollama e no mesmo hardware, os modelos qwen2.5:1.5b, qwen2.5:3b [12], phi4-mini [13] e gemma3:1b [6], cada um executado duas vezes consecutivas sobre uma única mão pré-flop (6♣ 6♦; A♠ 5♥ no caso do qwen2.5:1.5b), com um prompt curto e sem RAG. Nesses testes, o detector ainda era executado em formato PyTorch. A primeira execução inclui o carregamento do modelo na memória (partida a frio) e a segunda reflete o uso contínuo (partida a quente). A Tabela 3 resume os resultados.

Modelo	Primeira execução (s)	Demais execuções (s)	Ação
qwen2.5:1.5b	22,84	2,93	Raise
qwen2.5:3b	46,75	7,92	Raise
phi4-mini	66,35	27,77	Call
gemma3:1b	23,80	6,37	Call

Tabela 3 – Comparação de SLMs na Raspberry Pi 5 (decisão pré-flop).

O qwen2.5:1.5b foi o mais rápido, mas justificou a aposta apenas com "o Ás é a carta mais forte do deck". O qwen2.5:3b classificou o par de seis como "mão muito sólida", superestimando sua força. O phi4-mini foi o mais lento e, em uma das respostas, mencionou um flop inexistente. O gemma3:1b respondeu em 6,4 s na execução a quente e descreveu corretamente a mão como um par baixo, sendo escolhido pelo melhor equilíbrio entre latência e coerência. Como a comparação usou uma única mão e um prompt sem RAG, e o processamento do prompt domina a latência no cenário real (Seção 4.3), a ordem entre os modelos pode mudar com o prompt completo.

3.6 Base de conhecimento e RAG

A base de conhecimento foi construída a partir do livro "O livro verde do pôquer" [14]. O texto foi extraído do PDF, dividido em trechos de 300 caracteres com sobreposição de 60, convertido em vetores pelo modelo nomic-embed-text [15] e armazenado no Chroma, totalizando 11.129 trechos. Essa ingestão é feita uma única vez; durante o jogo, o sistema apenas consulta o índice. A consulta é escrita em português a partir da avaliação determinística (por exemplo, "estratégia no river com flush na posição Big Blind"), são recuperados os dois trechos mais similares e somente os que atingem relevância mínima de 0,35 entram no *prompt*, para evitar que conteúdo pouco relacionado contamine a resposta. Esse limiar foi definido empiricamente, a partir dos escores observados, sem otimização sistemática. Os prefixos de tarefa

recomendados para o nomic-embed-text (search_query: e search_document:) não foram utilizados, e o desempenho do modelo em português não foi avaliado. Depois, foi criada uma base curada de 43 fichas, uma por rua e situação, que resumem o livro com indicação de página; a ficha é recuperada por filtro de metadados, com esses prefixos, e só ela entra no *prompt*.

3.7 Camada de decisão híbrida

Antes de acionar o modelo de linguagem, o sistema verifica se a decisão já é conclusiva. Pelo caminho rápido (*fast-path*), mãos classificadas como "lixo_marginal" no pré-flop resultam em Fold, trincas ou jogos superiores resultam em Raise e carta alta sem projeto resulta em Check/Fold. Essas regras não são aplicadas quando a força está apenas na mesa ou quando há possibilidades claras de aumento de valor, casos em que o modelo pondera o cenário.

Nos demais casos, o *prompt* estruturado reúne a mão, as cartas da mesa, a posição, a análise determinística, os trechos recuperados, as diretrizes específicas de cada cenário e o formato obrigatório de resposta (AÇÃO e MOTIVO). A geração é limitada a 60 *tokens*, com temperatura 0,2, e o modelo permanece carregado na memória por 30 minutos após cada uso, sendo aquecido na inicialização do servidor. A resposta é interpretada e passa por uma verificação de coerência, que sinaliza, por exemplo, Fold com trinca ou superior e Raise com carta alta sem combinação provável.

3.8 Interface web

A aplicação Flask [16], acessível por qualquer dispositivo da rede local, organiza a validação em três passos: escolha da posição, registro das cartas em jogo e solicitação da decisão (Figura 2). As cartas podem vir da câmera (picamera2), de uma imagem salva previamente e carregada ou de digitação manual, o que permite corrigir erros de detecção antes de acionar o modelo. Para cada decisão, a interface exibe a imagem anotada, o caminho usado, a avaliação determinística, o tempo de cada módulo, a velocidade de geração e os trechos recuperados com seus tempos, além do *prompt* e da resposta bruta. Todas as decisões são registradas em um arquivo CSV com tempos, número de *tokens*, uso de CPU, memória e temperatura.

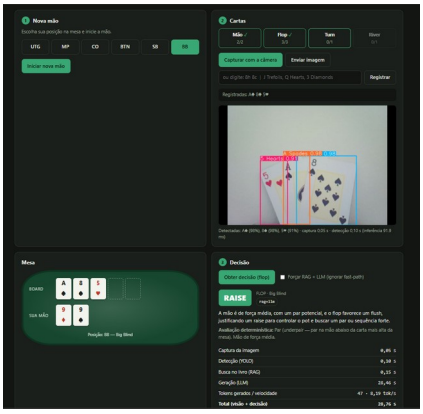


Figura 2 – Interface web durante a decisão no flop.

4 RESULTADOS

4.1 Protocolo

Todos os testes foram executados na Raspberry Pi 5, com o servidor Flask acessado por outro dispositivo da rede local. Foram avaliadas três mãos: (A) 7 e 2 de naipes diferentes no Button, decidida pelo caminho rápido FOLD; (B) 9♦ 9♠ no Big Blind, com todas as cartas capturadas pela câmera e mesa A♠ 8♠ 5♥ 3♠ 4♠, indicando RAISE e um cenário de *flush*; e (C) K♣ Q♣ em Under The Gun (UTG), com cartas enviadas por imagem ou digitadas e mesa A♣ 9♥ 2♦ 4♠ K♦, com o caminho rápido desativado, de modo que todas as decisões passaram pelo RAG e pelo modelo de linguagem. Também foram testados cenários de detecção incompleta ou excessiva. A amostra é pequena: três mãos e oito decisões com RAG, sem repetições. Cinco dessas decisões (as quatro da mão C e uma repetição do river da mão B) foram obtidas com o caminho rápido desativado e, portanto, não representam o fluxo normal do sistema.

4.2 Visão computacional

A inferência do YOLOv8n em NCNN levou entre 81,8 e 91,9 ms, e a detecção completa, incluindo pré e pós-processamento, entre 0,09 e 0,10 s. Nos testes da Seção 3.5, realizados antes da exportação para NCNN, a detecção com o modelo em formato PyTorch levava entre 283 e 346 ms, o que indica um ganho de aproximadamente três vezes com a exportação. A captura levou de 0,01 a 0,06 s, e as cartas foram identificadas com confiança entre 88% e 99%, inclusive quando parcialmente sobrepostas; a confiança, porém, não equivale à acurácia, que não foi medida separadamente nas capturas da mesa.

4.3 Pipeline completo

A Tabela 4 apresenta os tempos de decisão da mão B. Pelo caminho rápido, o river (flush) foi decidido sem acionar o modelo; na mão A, o Fold pré-flop levou 0,14 s da execução até a resposta, incluindo captura e detecção.

Rua	Caminho	RAG (s)	LLM (s)	Decisão (s)
Pré-flop	RAG+LLM	0,46	21,47	21,93
Flop	RAG+LLM	0,15	28,46	28,61
Turn	RAG+LLM	0,21	23,29	23,51
River	Fast-path	–	–	< 0,01
River	RAG+LLM	0,10	22,39	22,48

Tabela 4 – Tempos de decisão da mão B (9♦ 9♠, Big Blind). Decisão: tempo entre o pedido e a resposta, sem captura e detecção (0,09–0,10 s). –: etapa não executada.

A recuperação no Chroma levou de 0,09 a 0,46 s e não é um gargalo. O tempo do modelo de linguagem variou de 17,2 a 28,5 s nas oito decisões com RAG. Nas seis em que a velocidade foi registrada, o modelo gerou de 32 a 47 *tokens* a 8,2–9,1 *tokens*/s, o que corresponde a apenas 3,5–5,7 s de geração. Os 71% a 83% restantes do tempo, estimados por diferença, devem corresponder principalmente ao processamento do *prompt*, que inclui as instruções e os trechos do livro, o que o benchmark da Seção 4.5 confirma: os cerca de 140 *tokens* dos trechos acrescentaram 6,8 s ao modelo, com a mesma velocidade de geração. Isso explica por que o mesmo Gemma 3 1B responde em 6,4 s a um *prompt* curto (Tabela 3): a principal alavanca de otimização é o tamanho do *prompt*.

4.4 Comparação com a primeira etapa

Na primeira etapa, em que a geração não tinha limite de tamanho, as quatro decisões com RAG levaram em média 30,60 s (27,45 a 37,70 s); nesta etapa, as oito decisões levaram 22,11 s (17,63 a 28,61 s). As mãos testadas foram diferentes, de modo que a comparação é indicativa.

O tempo médio das decisões com RAG foi cerca de 28% menor que na primeira etapa. Entre as mudanças estão a limitação da geração, o modelo mantido em memória e o aquecimento na inicialização, mas seus efeitos não foram isolados; além disso, como o modelo gerou de 32 a 47 tokens com limite de 60, esse limite pode não ter atuado nesses testes. O maior ganho, porém, vem do caminho rápido, que elimina o modelo de linguagem nas situações conclusivas.

4.5 Qualidade das decisões e modos de falha

As decisões pelo caminho rápido seguem as regras por construção, o que garante consistência, mas não que as regras sejam as melhores para cada situação. O Fold com mãos "lixo_marginal" não considera que o Big Blind pode ter check grátis, e o Raise com flush na mão B não considera a textura da mesa: havia quatro espadas, e um adversário com K♠, Q♠, J♠ ou 10♠ teria flush superior ao do jogador. Na interface, a verificação de coerência sinalizou Raise com carta alta e sem projeto na mão C (Figura 3, Apêndice A).

Para medir a qualidade das decisões, um *script* na Raspberry Pi sorteou 40

situações (dez por rua, semente fixa), cada uma decidida pelo Gemma 3 1B com o caminho rápido desativado, variando apenas o conteúdo recuperado. Como referência, a equidade contra dois oponentes com mãos aleatórias foi estimada por Monte Carlo e convertida em ação: Raise acima de 1,5 vez a fatia justa de 1/3, Fold abaixo de 0,75 vez e Call/Check entre esses limites. Erro grave é um Raise com referência Fold, ou o contrário (Tabela 5). A rodada 1 (semente 2026) comparou o modelo com e sem o livro; a rodada 2 (semente 2027, situações novas) incluiu a base curada, a ação corrigida, em que a resposta incoerente é trocada pela ação segura da regra (com novas regras para jogo só na mesa e sequência interna no turn), e o sistema híbrido, com o caminho rápido.

Métrica	Sem RAG	Livro	Curada
R1: LLM	55,0/15,0	30,0/35,0	–
R2: LLM	42,5/17,5	37,5/22,5	47,5/12,5
R2: corrigida	50,0/10,0	50,0/10,0	50,0/10,0
R2: híbrido	52,5/10,0	52,5/10,0	52,5/10,0
R2: tempo (s)	10,0	17,3	12,1

Tabela 5 – Concordância/erros graves (%) frente à referência e tempo médio de decisão nas rodadas 1 (R1) e 2 (R2).

Na primeira rodada, o livro piorou as decisões: em 11 situações apenas a versão sem RAG concordou com a referência, e em uma ocorreu o contrário (teste exato de McNemar [17], $p \approx 0{,}006$). Com 9♥ 5♣ no river e o par apenas na mesa (equidade de 0,6%), o modelo desistiu sem RAG, mas apostou com RAG porque "a carta do river é um kicker forte". O livro adota um estilo agressivo [14], e o conselho de apostar sem jogo, fora de contexto, vira recomendação de aposta. O limiar de 0,35 não filtrou nenhum trecho (escores de 0,50 a 0,79).

Na segunda rodada, as diferenças entre as fontes foram menores e não significativas ($p \geq 0{,}34$). A base curada teve a maior concordância do modelo (47,5%), menos erros graves e respostas incoerentes, inverteu o viés do livro (14 Raise contra 29) e levou 12,1 s por decisão, 30% menos que com o livro. O maior ganho veio da correção, que levou as três fontes a 50% de concordância e 10% de erros graves; com o caminho rápido, o sistema híbrido chegou a 52,5%. A base curada ainda errou ao desistir com pares médios no turn, e a estimativa de 72,5% feita a posteriori na rodada 1 não se repetiu em situações novas.

A referência ignora pote, apostas e histórico da mão, e seus limites foram definidos pelo autor; as situações estão nas Tabelas 6 e 7 (Apêndice B). Nos cenários de detecção, o sistema tratou corretamente cartas a menos (aviso e pedido de nova captura ou digitação), cartas a mais (mantendo as de maior confiança) e cartas já registradas na rodada (ignoradas).

4.6 Uso de recursos

Com o detector, o índice e o modelo carregados, o sistema ocupou cerca de 35% da memória, com temperatura de 54,5 °C em repouso. Durante a inferência do modelo de linguagem, os quatro núcleos operaram próximos de 100%, confirmando que a etapa é limitada pela CPU. No benchmark, com 19 minutos de inferência contínua, a temperatura máxima foi de 73,2 °C.

5 CONCLUSÃO

O sistema desenvolvido executa localmente, na Raspberry Pi 5, todo o fluxo da imagem à recomendação estratégica, cobrindo o cenário de uma mão e a posição do jogador. A detecção com YOLOv8n em NCNN leva cerca de 0,1 s com mAP50 de 0,994 no conjunto de teste, a interface Flask permitiu validar o sistema de ponta a ponta e registrar métricas de cada módulo, e a arquitetura híbrida mostrou-se essencial: o caminho rápido responde de forma imediata nas situações conclusivas, nos demais casos o modelo decide a partir da avaliação determinística da mão e a verificação determinística de coerência sinaliza respostas contraditórias.

Assim como observado por Carmo Junior et al. [8], a inferência do SLM é o componente dominante da latência. As decisões com RAG levaram de 17 a 29 s, e a adequação desse tempo ao ritmo de uma partida real ainda precisa ser avaliada. No benchmark, o livro reduziu a concordância com a referência de 55% para 30%; em situações novas, a base curada teve o melhor resultado do modelo (47,5%), com decisões 30% mais rápidas, e a correção automática levou o sistema híbrido a 52,5%, com 10% de erros graves, com qualquer fonte: a qualidade da decisão depende mais da camada determinística do que do RAG. Os resultados se limitam a uma referência aproximada, não avaliam o detector em capturas novas da mesa e não incluem o consumo de energia.

Como trabalhos futuros, destacam-se a avaliação contra um solver ou especialista; regras para sequência interna no flop; a avaliação do detector em capturas novas da mesa; a medição de consumo; a redução do *prompt* (menos trechos, chunking estrutural e uma tabela pré-computada das 169 mãos pré-flop por posição), o uso de aceleradores de hardware, a avaliação do YOLO11n com quantização int8 e a inclusão de oponentes, pote e pilhas de fichas.

REFERÊNCIAS

[1] ZHOU, Z. et al. Edge intelligence: paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE, v. 107, n. 8, p. 1738-1762, 2019.

[2] ROVAI, M. Edge AI engineering: hands-on with the Raspberry Pi. [S. l.], 2025. Disponível em: https://mjrovai.github.io/EdgeML_Made_Ease_ebook/. Acesso em: 25 set. 2026.

[3] BILLINGS, D. et al. The challenge of poker. Artificial Intelligence, v. 134, n. 1-2, p. 201-240, 2002.

[4] RUBIN, J.; WATSON, I. Computer poker: a review. Artificial Intelligence, v. 175, n. 5-6, p. 958-987, 2011.

[5] BROWN, N.; SANDHOLM, T. Superhuman AI for heads-up no-limit poker: Libratus beats top professionals. Science, v. 359, n. 6374, p. 418-424, 2018.

[6] GEMMA TEAM. Gemma 3 technical report. arXiv:2503.19786, 2025. Disponível em: https://arxiv.org/abs/2503.19786.

[7] LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 33., 2020. Proceedings [...]. Red Hook: Curran Associates, 2020. p. 9459-9474.

[8] CARMO JUNIOR, R. M. et al. An edge AI architecture for privacy-preserving home automation using small language models. In: INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE, COMPUTER, DATA SCIENCES AND APPLICATIONS (ACDSA), 2027, Rio de Janeiro. [Aceito para publicação / No prelo].

[9] JOCHER, G.; CHAURASIA, A.; QIU, J. Ultralytics YOLOv8. Versão 8.0.0. 2023. Disponível em: https://github.com/ultralytics/ultralytics.

[10] JOCHER, G.; QIU, J. Ultralytics YOLO11. Versão 11.0.0. 2024. Disponível em: https://github.com/ultralytics/ultralytics.

[11] JOCHER, G. et al. Ultralytics YOLO26: unified real-time end-to-end vision models. arXiv:2606.03748, 2026.

[12] QWEN TEAM. Qwen2.5 technical report. arXiv:2412.15115, 2024. Disponível em: https://arxiv.org/abs/2412.15115.

[13] ABOUELENIN, A. et al. Phi-4-Mini technical report: compact yet powerful multimodal language models via mixture-of-LoRAs. arXiv:2503.01743, 2025. Disponível em: https://arxiv.org/abs/2503.01743.

[14] GORDON, P. O livro verde do pôquer: Texas Hold'em. São Paulo: Marco Zero, 2007.

[15] NUSSBAUM, Z. et al. Nomic Embed: training a reproducible long context text embedder. arXiv:2402.01613, 2024. Disponível em: https://arxiv.org/abs/2402.01613.

[16] PALLETS. Flask documentation. 2024. Disponível em: https://flask.palletsprojects.com/. Acesso em: 1 out. 2026.

[17] McNEMAR, Q. Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, v. 12, n. 2, p. 153-157, 1947.

APÊNDICE A – CAPTURAS DA INTERFACE



Figura 3 – Alerta de incoerência gerado pela verificação determinística.

APÊNDICE B – SITUAÇÕES DO BENCHMARK

Situações sorteadas nas duas rodadas (sementes 2026 e 2027) e decididas pelo Gemma 3 1B (Seção 4.5). Equidade estimada por Monte Carlo contra dois oponentes com mãos aleatórias (3.000 simulações). C/C: Call/Check. *: erro grave (Raise com referência Fold, ou o contrário). Ações do modelo antes da correção. Na rodada 2, a situação 33 (mão especulativa) recuperou uma ficha de outra categoria, pois as fichas dessa categoria foram incluídas depois.

Nº	Rua	Pos.	Mão	Mesa	Equid.	Ref.	Sem RAG	Com RAG
1	Pré	SB	3♣ 7♥	–	0,22	Fold	Fold	Fold
2	Flop	SB	10♥ Q♦	3♠ 5♠ J♠	0,16	Fold	Raise*	Raise*
3	Turn	SB	10♣ 8♠	J♥ A♣ K♠ 9♣	0,23	Fold	Fold	Raise*
4	River	UTG	9♥ 5♣	2♥ J♣ 3♦ 3♣ 6♠	0,01	Fold	Fold	Raise*
5	Pré	BB	9♥ 2♥	–	0,26	C/C	Fold	Raise
6	Flop	CO	9♣ Q♣	7♥ 5♦ 8♦	0,27	C/C	Raise	Raise
7	Turn	UTG	7♠ A♣	8♥ K♣ 10♥ Q♥	0,18	Fold	Raise*	Raise*
8	River	BTN	K♠ 7♦	3♦ 10♣ 10♠ 6♠ A♣	0,21	Fold	Fold	Fold
9	Pré	BB	4♦ Q♦	–	0,34	C/C	Fold	Fold
10	Flop	CO	K♦ J♥	6♣ 2♦ 7♣	0,22	Fold	Raise*	Raise*
11	Turn	BTN	9♦ 8♣	3♦ 8♦ J♦ 10♣	0,51	Raise	Raise	Raise
12	River	SB	3♣ 8♣	10♥ A♥ J♠ 9♣ 8♦	0,06	Fold	Fold	Raise*
13	Pré	SB	6♥ 8♠	–	0,28	C/C	Fold	Raise
14	Flop	SB	9♠ 10♥	10♦ A♣ 2♦	0,52	Raise	Raise	Raise
15	Turn	BB	5♠ A♦	4♥ 2♣ K♦ 10♦	0,26	C/C	Raise	Raise
16	River	MP	3♠ K♦	J♣ 8♣ 9♠ 4♦ K♣	0,62	Raise	C/C	Raise
17	Pré	BB	3♣ Q♥	–	0,30	C/C	Fold	Fold
18	Flop	CO	9♦ Q♣	9♠ K♦ 10♠	0,50	C/C	Raise	Raise
19	Turn	SB	10♣ 8♦	Q♠ 6♣ A♣ Q♦	0,16	Fold	Fold	Raise*
20	River	UTG	K♥ 9♠	8♦ 8♣ 5♥ 6♠ 6♦	0,30	C/C	C/C	Raise
21	Pré	BB	2♠ 4♦	–	0,20	Fold	Fold	Raise*
22	Flop	CO	J♠ 9♦	A♣ 10♦ 9♣	0,48	C/C	Raise	Raise
23	Turn	BTN	5♥ 6♥	K♥ 9♣ 9♦ 10♥	0,24	Fold	Fold	Raise*
24	River	BTN	A♣ 8♣	10♣ Q♠ 2♠ 6♠ 7♥	0,09	Fold	Fold	Fold
25	Pré	BTN	A♣ 3♠	–	0,37	C/C	Fold	Fold
26	Flop	BTN	K♥ K♦	A♠ 10♥ 6♥	0,63	Raise	Raise	Raise
27	Turn	CO	4♣ 8♣	K♠ 9♠ 3♥ Q♦	0,05	Fold	Fold	Raise*
28	River	BTN	9♣ 7♣	3♠ 5♥ K♣ 8♠ 4♦	0,00	Fold	Fold	Fold
29	Pré	UTG	2♥ 3♦	–	0,20	Fold	Fold	Raise*
30	Flop	MP	9♥ J♦	10♠ 3♦ J♣	0,65	Raise	Raise	Raise
31	Turn	BB	5♣ 3♣	J♣ 2♠ 10♠ 6♦	0,13	Fold	Raise*	Raise*
32	River	SB	A♥ Q♥	7♠ 4♦ 2♥ 7♥ Q♠	0,81	Raise	Fold*	C/C
33	Pré	BTN	J♥ A♦	–	0,45	C/C	Raise	Raise
34	Flop	BTN	K♥ K♠	4♥ 10♠ 5♠	0,68	Raise	Raise	Raise
35	Turn	UTG	J♦ 4♥	J♠ K♥ 10♥ 6♠	0,47	C/C	C/C	Raise
36	River	BTN	8♠ Q♠	4♠ 4♥ J♦ 6♣ 10♥	0,07	Fold	Fold	Fold
37	Pré	MP	8♥ A♣	–	0,40	C/C	Raise	Raise
38	Flop	UTG	10♦ A♣	5♥ 8♥ 2♥	0,22	Fold	Raise*	Raise*
39	Turn	UTG	8♥ 3♥	2♣ 7♠ Q♦ A♥	0,05	Fold	Fold	Raise*
40	River	CO	2♦ 10♥	9♠ 7♥ Q♣ 6♣ 9♥	0,02	Fold	Fold	Fold

Tabela 6 – Situações da rodada 1, ação de referência e ação do modelo com e sem RAG.

Nº	Rua	Pos.	Mão	Mesa	Equid.	Ref.	Sem RAG	Livro	Curada
1	Pré	UTG	3♠ 9♦	–	0,24	Fold	Fold	Fold	Fold
2	Flop	BB	2♥ Q♦	7♥ A♠ 8♦	0,15	Fold	Raise*	Raise*	Fold
3	Turn	MP	A♣ 4♣	8♦ A♠ 7♠ J♠	0,53	Raise	Raise	Raise	C/C
4	River	CO	6♦ 6♠	Q♣ J♣ 3♥ Q♥ Q♠	0,62	Raise	Raise	Raise	Raise
5	Pré	CO	6♣ 3♥	–	0,22	Fold	Fold	Fold	Fold
6	Flop	MP	A♥ Q♦	6♥ 8♣ 10♥	0,31	C/C	Raise	Raise	Fold
7	Turn	BTN	4♠ K♠	7♦ 7♠ 5♠ 8♦	0,35	C/C	Fold	Raise	Fold
8	River	UTG	8♥ 4♠	9♦ 7♣ K♣ 7♥ J♣	0,01	Fold	Fold	Fold	Fold
9	Pré	UTG	A♦ 5♥	–	0,39	C/C	Raise	Raise	Raise
10	Flop	CO	7♥ 9♠	6♥ 10♦ 3♠	0,28	C/C	Raise	Raise	Raise
11	Turn	CO	A♥ 8♠	8♥ J♥ 3♦ 4♣	0,58	Raise	Raise	Raise	Fold*
12	River	BTN	J♦ 3♥	3♦ Q♦ J♣ 10♣ 10♥	0,45	C/C	Raise	Raise	C/C
13	Pré	MP	3♥ 9♣	–	0,23	Fold	Fold	Raise*	Fold
14	Flop	BB	5♦ 2♠	3♣ 6♣ J♦	0,19	Fold	Raise*	Raise*	Raise*
15	Turn	SB	A♥ 10♥	Q♠ 3♦ 10♠ 2♦	0,57	Raise	Fold*	Raise	Fold*
16	River	MP	Q♥ 3♣	J♣ 4♥ 7♦ 9♥ 2♥	0,01	Fold	Fold	Fold	Fold
17	Pré	BB	2♦ 2♥	–	0,31	C/C	Raise	Raise	Raise
18	Flop	SB	7♦ Q♥	7♣ 10♥ 6♠	0,46	C/C	Raise	Raise	Raise
19	Turn	MP	8♥ 9♠	9♦ 5♦ A♣ A♦	0,50	Raise	Raise	Raise	Raise
20	River	CO	9♠ 6♦	5♣ J♠ 2♦ A♣ K♥	0,02	Fold	Fold	Fold	Fold
21	Pré	SB	Q♥ Q♠	–	0,66	Raise	Raise	Raise	Raise
22	Flop	BB	10♣ 4♦	5♦ 7♣ K♣	0,16	Fold	Raise*	Raise*	Fold
23	Turn	BTN	Q♠ 5♠	3♠ 3♦ 6♣ 9♣	0,13	Fold	Fold	Fold	Fold
24	River	BB	4♥ Q♠	6♥ 9♥ K♣ A♣ A♦	0,21	Fold	Fold	Fold	Fold
25	Pré	CO	J♠ 8♣	–	0,33	C/C	Fold	Fold	Fold
26	Flop	BB	7♠ 5♣	J♥ 10♦ 8♠	0,15	Fold	Raise*	Raise*	Raise*
27	Turn	BB	10♦ 2♦	K♣ 5♥ 9♦ 4♥	0,06	Fold	Fold	Raise*	Fold
28	River	CO	6♠ 4♠	4♦ 8♦ J♦ 6♣ 5♦	0,19	Fold	Raise*	Raise*	C/C
29	Pré	MP	7♠ J♦	–	0,30	C/C	Fold	Fold	Fold
30	Flop	CO	J♣ 8♥	6♥ Q♣ 6♠	0,21	Fold	Raise*	Raise*	Raise*
31	Turn	UTG	5♥ 3♠	A♠ 5♠ 6♥ 8♠	0,30	C/C	Raise	Raise	Fold
32	River	BTN	2♥ 9♦	2♠ A♥ K♣ 4♥ A♣	0,31	C/C	Raise	Raise	C/C
33	Pré	MP	Q♠ J♦	–	0,41	C/C	Raise	Raise	Raise
34	Flop	UTG	6♣ J♥	6♦ 10♠ 8♦	0,37	C/C	Raise	Raise	Raise
35	Turn	MP	Q♦ 3♦	10♠ 8♠ 6♥ 3♥	0,27	C/C	Raise	Raise	Fold
36	River	UTG	8♥ 10♦	J♥ 7♥ Q♦ 5♥ 4♦	0,02	Fold	Fold	Fold	Fold
37	Pré	BB	J♣ 4♠	–	0,26	C/C	Fold	Fold	Fold
38	Flop	BB	7♦ 8♠	8♥ K♦ 4♥	0,47	C/C	Raise	Raise	Raise
39	Turn	SB	2♣ 4♣	2♥ J♥ 9♥ K♦	0,21	Fold	Fold	Raise*	Fold
40	River	BTN	7♦ 5♥	4♣ A♥ 6♠ 3♠ 5♦	0,86	Raise	Raise	Raise	Raise

Tabela 7 – Situações da rodada 2, ação de referência e ação do modelo por fonte de conhecimento.

APÊNDICE C – TREINAMENTO E AVALIAÇÃO DO DETECTOR

Resultados do treinamento do YOLOv8n por 50 épocas no Google Colab (GPU Tesla T4, Ultralytics 8.4), com imagens de 640 × 640 pixels, lotes de 16 e semente 0. O melhor peso (best.pt) corresponde à época 44. Este treinamento, feito para avaliar os três conjuntos, repetiu a configuração do modelo embarcado na Raspberry Pi, que havia atingido mAP50 de 0,993 e mAP50-95 de 0,983 na validação.

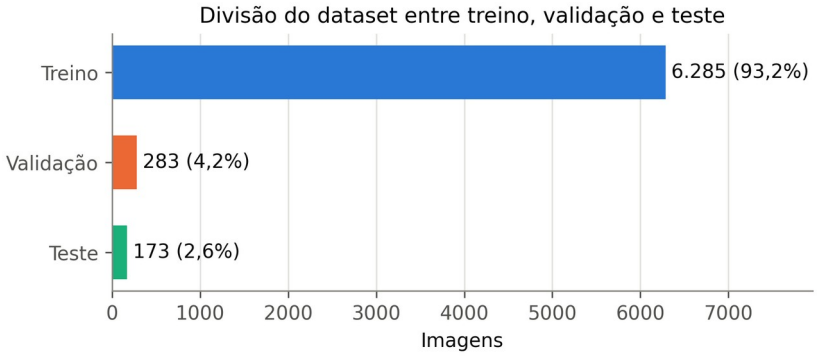


Figura 4 – Divisão das imagens entre treino, validação e teste; o treino inclui as versões geradas por aumento de dados.

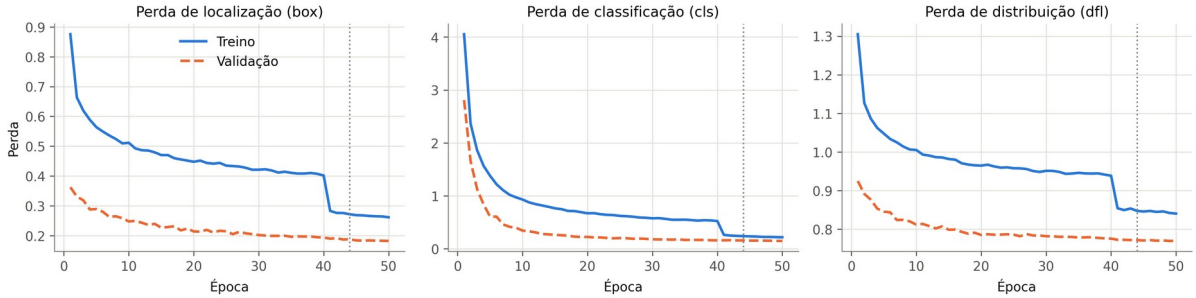


Figura 5 – Perdas de treino e validação por época. A queda na época 41 coincide com o desligamento do mosaico nas últimas dez épocas, padrão do Ultralytics; a linha pontilhada marca o melhor peso.

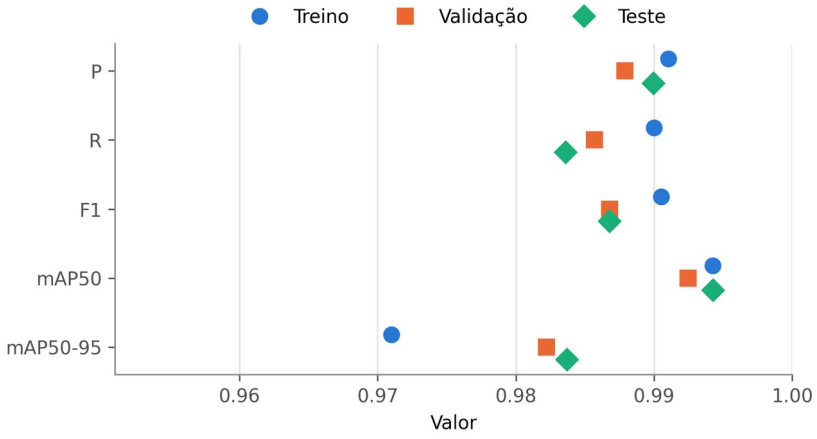


Figura 6 – Métricas do YOLOv8n (50 épocas) nos conjuntos de treino, validação e teste.

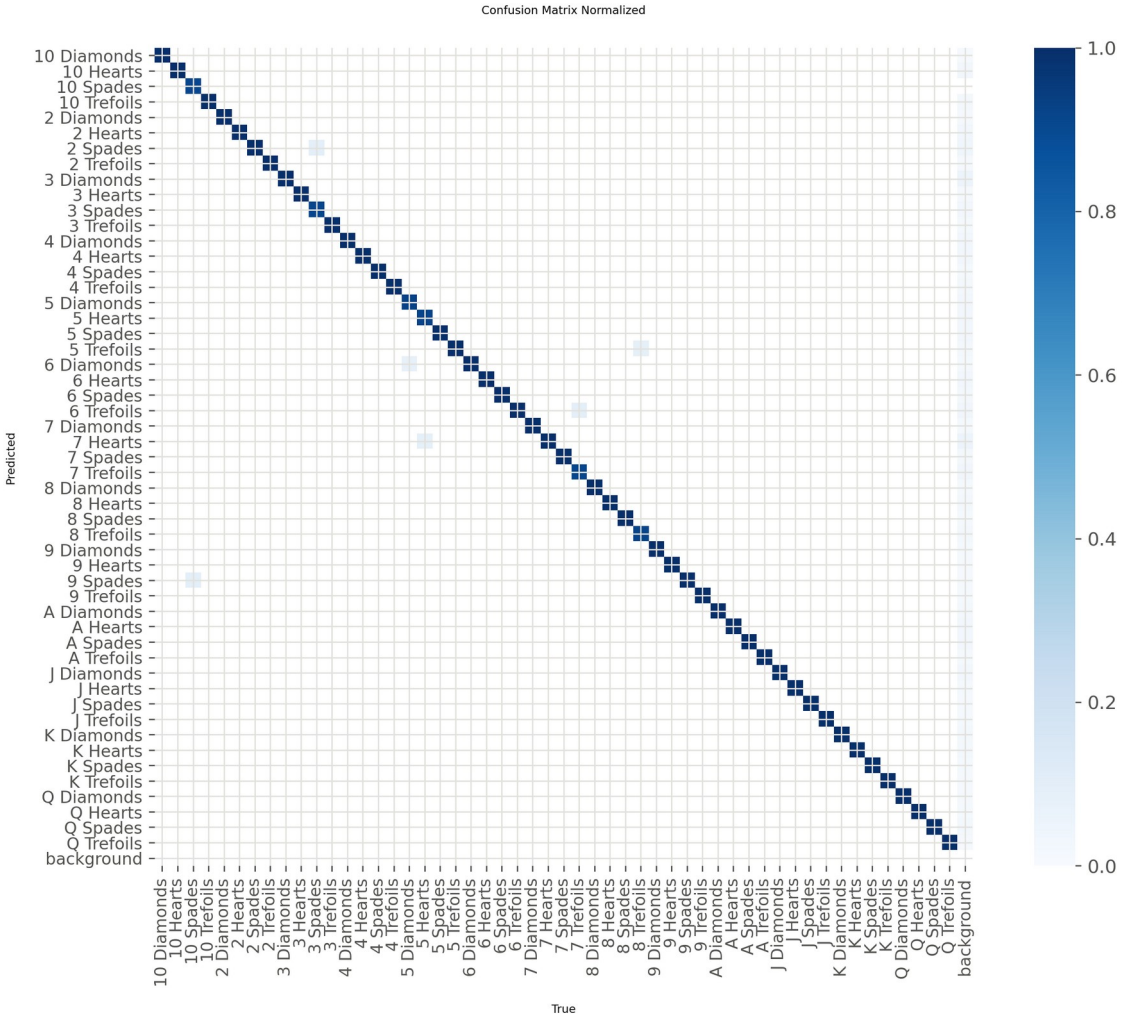


Figura 7 – Matriz de confusão normalizada do YOLOv8n (50 épocas) no conjunto de teste.